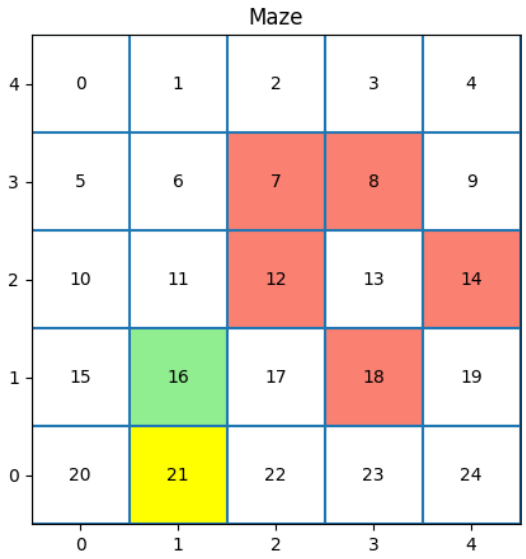
Co-Lab Link:
https://colab.research.google.com/drive/16YGnadIpPMMAgc_oefE2cwrKwVBMtMRC#scrollTo=wgfhuESJiJgW

Maze Description:



The grid world I designed for this project is a 5x5 matrix with random start, goal, and wall cells, which are visualized in the initial maze layout. Each cell represents a unique state. The agent starts in a specific cell that is highlighted by yellow and aims to reach the goal cell which is highlighted in green. Red cells represent walls, which the agent cannot traverse, forcing it to find alternative paths.

Problem Formulation:

Each cell in the 5x5 grid represents a unique state, total of 25 states. The agent can take one of four possible actions at each step: move up, down, left, or right. If the agent encounters a wall or moves outside the grid boundary, it remains in the same cell. As for the rewards a step towards the goal receives a small negative reward to encourage efficient paths. Reaching the goal state provides a high positive reward. Moving into walls or boundaries does not yield any reward but counts as a step.

Q Network Design:

The Q-network approximates the Q-values for each state-action pair and is implemented as a neural network with three fully connected layers Input Layer, where it receives a flattened representation of the maze state and goal states. Hidden Layers where the first hidden layer consists of 150 neurons, and the second layer has 100 neurons, each using the ReLU activation function. And for the output layer it consists of 4 neurons, each representing the Q-value for one of the four possible actions. The network is trained using the mean squared error (MSE) between the predicted Q-values and target values, with an Adam optimizer to minimize the loss over episodes.

Pseudo Code:

Below is the pseudo code summarizing the Q-learning with a neural network (QNN) for this maze setup:

Step 1: Initializing the Q-network with random weights.

Step 2: Defining the maze environment with start, goal, and wall cells.

Step 3: a. For each episode:

b. Resetting the agent to the starting position.

c. When the goal is not reached:

Choosing an action using epsilon-greedy policy.

Observing reward and new state from the environment.

Calculating target Q-value:

If the new state is the goal, then the target = reward.

Otherwise, target = reward + (discount factor * max Q for next state).

Training Q-network on the observed state and action.

Updating the agent's position.

Decay epsilon to reduce exploration over time.

Results:

Mean Squared Error (MSE)

The Mean Squared Error (MSE) acts as the goal for the Q-network, measuring the disparity between the anticipated Q-values and the target values obtained from the Q-learning update regulation. At the beginning, the Mean Squared Error is elevated because of the random start of network weights and the agent's inexperience with the environment. As the training advances, the Q-network improves its predictions by incorporating feedback after each episode. This process leads to a gradual reduction in MSE, as depicted in the graph of Loss versus Episodes.

Initial Phase from Episodes 0 to 20: The MSE fluctuates significantly due to the agent's exploratory actions. This phase sees large prediction errors as the Q-network starts adapting to the environment and learning which actions lead to higher rewards.

From Episodes 20 to 60: The MSE begins to stabilize, displaying a downward trend as the Q-network fine-tunes its predictions for state-action pairs. The fluctuations reduce as the agent balances exploration and exploitation, gradually focusing more on optimal actions.

From Episodes 60-100: The MSE converges to near-zero, indicating that the Q-network has successfully learned the optimal Q-values for each state-action pair. At this stage, the Q-network has minimized prediction errors, signaling that the agent has achieved a stable policy for reaching the goal.

This steady decrease in MSE over episodes reflects the Q-network's increasing accuracy in approximating the Q-values, which enables the agent to navigate the maze effectively.

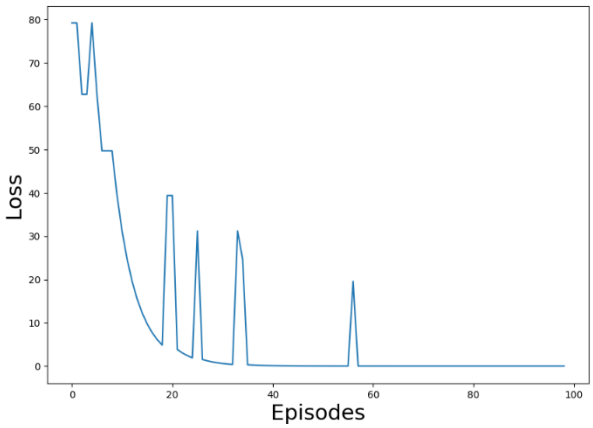
Network Weights

The network weights undergo significant adjustments in the early training stages as the Q-network attempts to learn the relationships between states and actions. During the initial episodes, large weight updates occur, reflecting the Q-network's adaptation to the feedback received from the environment. These weight updates are progressively smaller as training advances, indicating convergence.

At the beginning large weight adjustments occur as the network begins learning from scratch. These early updates are essential for the Q-network to form rough approximations of Q-values. After that the weight adjustments become more refined as the Q-network gets closer to the true Q-values. The gradients for weight updates decrease, reflecting more accurate approximations and a reduced need for drastic changes. Finally, the weight changes are minimal as the Q-network reaches convergence. At this point, weights are close to their optimal values, allowing the Q-network to produce stable Q-values for guiding the agent's actions.

This convergence in network weights is vital for stability in the policy, as drastic changes in weights could lead to inconsistencies in action selection. The gradual decrease in weight updates aligns with the reduced MSE, showing that the Q-network has effectively learned to approximate the optimal Q-values.

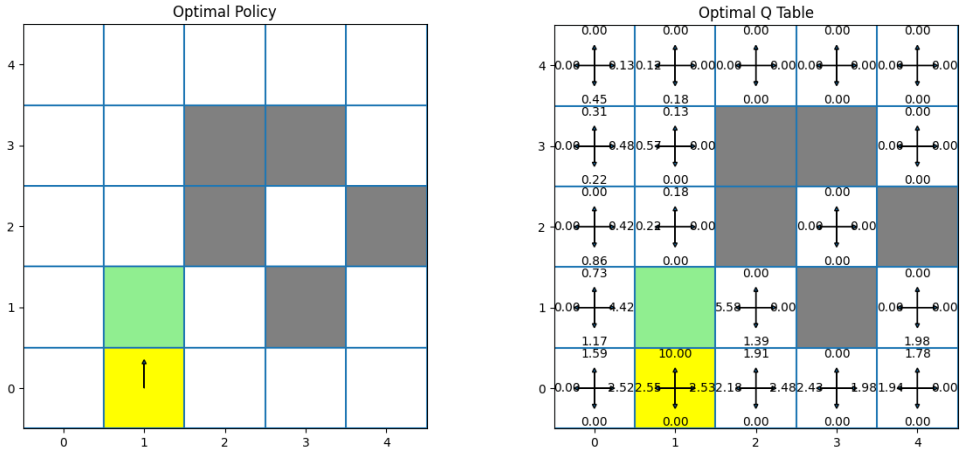
Convergence of the Loss Function



The training process was evaluated over multiple episodes, with the loss function plotted against each episode. Initially, the agent exhibits a high loss value, reflecting the neural network's inexperience in accurately predicting Q-values for each state-action pair. Over time, as the agent interacts with the environment, the Q-network begins to learn the optimal actions, resulting in a gradual reduction in the loss. By around episode 60, the loss reaches near-zero values, indicating that the Q-network has learned the optimal policy for navigating the maze.

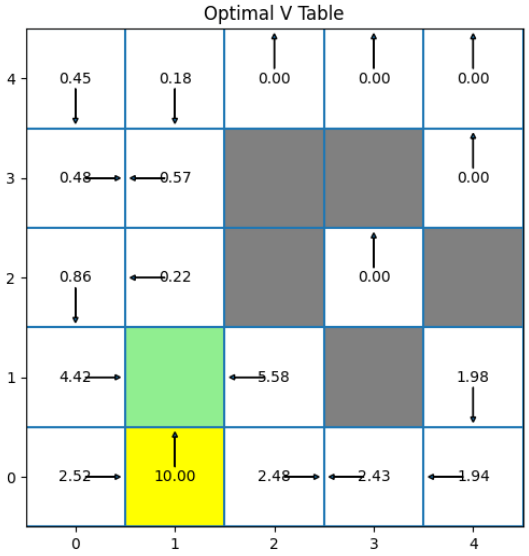
The convergence pattern of the loss function, as shown in the graph, displays oscillations at the beginning, typical of Q-learning due to exploration. The reduction in loss and subsequent stabilization as episodes progress suggests that the Q-network has successfully minimized prediction errors, thus achieving a stable and effective policy.

Optimal Policy and Q-Values



The optimal policy obtained from the final Q-table clearly guides the agent from the starting position which is yellow cell towards the goal position in green cell. The policy arrows on the grid indicate the best action at each cell, ensuring that the agent follows an efficient path to reach the goal without encountering walls. The Q-table values further support this, with higher Q-values observed for state-action pairs that move the agent closer to the goal state.

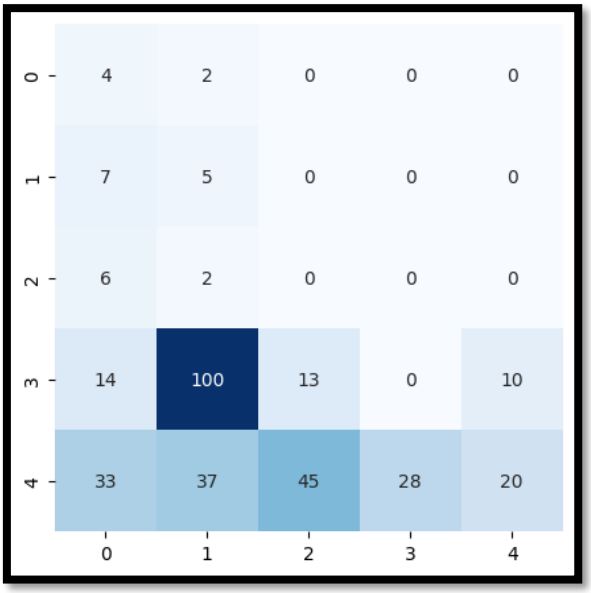
The visualization of Q-table shows that non-zero values concentrated in the states that helps to form the optimal path, while states away from the goal have values close to zero, as they are not part of the agent's optimal policy. This highlights that the agent has learned to prioritize its actions that give a yield higher cumulative rewards, aligning with the goal-directed behaviour.



Optimal V-Values

The optimal V-values table captures the maximum expected reward achievable from each state under the learned policy. Similar to the Q-table, higher V-values are detected for states on or near the optimal path, particularly the start and goal states. This outcome indicates that the agent has successfully internalized the goal-oriented nature of the environment, as states closer to the goal inherently possess higher utility.

Heatmap of States visited:



The heatmap provides an additional insight into the agent's exploration and learning process. The heatmap shows a high concentration of visits in states along the optimal path, particularly around the start and goal areas. This suggests that as the training progresses, the agent becomes more focused on the optimal path, spending less time exploring suboptimal paths.

The heatmap also demonstrates how the exploration strategy gradually gives way to exploitation as the agent refines its policy. States with fewer visits, often in boundary cells or cells obstructed by walls, were quickly disregarded by the agent once they were identified as non-beneficial for reaching the goal.

References

1. https://github.com/Eligijus112/rl-snake-game
2. https://towardsdatascience.com/part-1-building-a-deep-q-network-to-play-gridworld-deepminds-deep-q-networks-78842007c631
3. Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.